Constant Change

A scheduling mode for quad-box — how permutations are chosen, how long they last, and how the n-level carries across. Written against the integrated build.

The mode runs two clocks at once. The **short-term** clock changes the texture of every session: feedback, rotation, interference, trial count. The **long-term** clock changes what you are actually training — the combination of modalities and stimulus sets — and only moves when performance stops climbing. The aim is that no two training days are alike, and that no permutation stays long enough to become comfortable.

Both clocks draw from exhaustive no-repeat pools: an option cannot come up again until every other option in its pool has been used. When a pool empties it refills and the cycle restarts.

1. The permutation space

A **permutation** is the long-term identity of a training block. It fixes which modalities are running and which stimulus set each one draws from. It does not fix difficulty or session texture — those belong to the short-term clock.

Component	Chosen from	Notes
Position	2D grid or 3D grid	Re-rolled for every block, never fixed to one
Audio	Any enabled audio set	Always present in every tier
Visual	Colour, shape or image — and a specific set within it	None in dual; one in tri; colour + shape in quad

What is actually available

The catalog in `constantChange.js` mirrors quad-box's own option lists exactly, so the two cannot drift apart:

Channel	Sets	Count
Grid	3D, 2D	2
Audio	Letters M1, M2, F1, F2, F3, Numbers, NATO, 5 syllables, 10 syllables	9
Colour	Basic, Gradient, Voronoi, Generative Art	4
Shape	Basic, Tetris, Icons A, Icons B, All Shapes	5
Image	Voronoi, Generative Art	2

Tier sizes

Tier	Formula	Size
dual	grid × audio	2 × 9 = 18
tri	grid × audio × (colour + shape + image sets)	2 × 9 × 11 = 198
quad	grid × audio × colour × shape	2 × 9 × 4 × 5 = 360
	total	576

Why quad is colour + shape only

This is a rendering constraint in quad-box, not a design choice. A cell has exactly two visual channels: a background colour and a background image. In `trialUtils.js`, `createSvgId()` returns immediately when an image is present and `findBoxColor()` returns empty, so an image drawn beside a colour or a shape hides it completely — while the game still generates and scores that hidden stimulus, asking the player to track something they cannot see. Colour + shape is the only pair that renders, and it is exactly what quad-box already calls quad. Image therefore appears at tri only. `QUAD_PAIRS` encodes this in one line if the renderer ever gains a third channel.

Each tier keeps its own independent pool. Drawing a tri permutation does not consume anything from the dual or quad pools.

2. The tier cycle

Blocks advance through the tiers in a fixed loop. Every plateau moves the cursor one step:

```
dual → tri → quad → dual → ...
```

So the load climbs from two simultaneous streams to four, then resets to two with a higher n-level. Within each step, *which* permutation you get is a fresh no-repeat draw from that tier's pool — position and audio are re-rolled every time, so returning to dual does not mean returning to the same dual.

3. Short-term: what changes every day

Each training day draws one variant. Four axes vary:

Axis	Values	Count
Feedback	shown / hidden	2
Rotation speed	0 / 200 — 3D grid only	2 (or 1)
Interference	10% / 25% / 40%	3
Trials	40 / 100 / 150 / 200	4

Rotation speed is meaningless on a 2D grid, so that axis collapses rather than wasting half the pool on a duplicate. A 3D permutation therefore has **48** distinct day-variants and a 2D permutation has **24**.

The variant pool is scoped to the current permutation and **resets whenever the permutation changes**. Within one block, no variant repeats until all of them have been seen — which in practice never happens, since blocks are far shorter than 24 days.

The draw is made **once per training day** and held for every session that day. That is deliberate: the plateau test compares one day's accuracy against another's, and that comparison is only fair if trial count and

interference stayed put in between. Set `variantScope` to `session` in the panel to redraw every game instead.

A training day starts at 4am, reusing the app's own `getGameDay()`, so a late-night session counts towards the day it began.

4. Long-term: when a permutation ends

The block ends on a plateau. A **training day** is one day with at least one completed session; days with no session are simply absent, so "three days in a row" means three consecutive entries in the log, not three consecutive dates.

The log is **rebuilt from the game database on every sync** rather than accumulated as it goes. Nothing is double-counted if a hook fires twice, a missed session is picked up on the next sync, and clearing your history clears the schedule with it.

Where a day contains several sessions, the day's n-level is the **highest n reached** and the day's accuracy is the **mean across those sessions**.

The rule

```
IF the last 3 training days share an identical n-level
AND day 3 accuracy < day 1 accuracy × 1.15
THEN the permutation changes.
```

Both conditions are required. If n moved at all across the window — up or down — the streak is broken and nothing happens. A falling n-level is explicitly allowed and never forces a change on its own.

Threshold reading. "15% better" is implemented as a *relative* gain (60% must reach 69%). An absolute reading is available as a config switch (`improvementMode: 'absolute'`), where 60% must reach 75%. Relative is the default.

One consequence is worth stating plainly: at high accuracy the bar becomes unreachable. A player sitting at 90% would need 103.5%, so the plateau always fires. This is the correct behaviour — 90% accuracy at a stable n-level *is* mastery, and mastery is exactly when the permutation should change.

5. Long-term: choosing the next permutation

Once a plateau fires, the tier cursor advances and a permutation is drawn from the new tier's pool, excluding anything already used since that pool last refilled. When the pool is exhausted it refills; on a refill the immediately-previous permutation for that tier is skipped, so a reset never surfaces as a back-to-back repeat.

Both position and audio are drawn fresh, and for tri and quad the extra family *and* the specific set within it are drawn fresh too.

6. The n-level handoff

The new block does not start from scratch. It inherits the **highest n reached during the block that just ended**, scaled by the change in simultaneous load:

```
n_next = floor( n_peak × modalities_from / modalities_to )
```

Transition	Scaling	Example (n_peak = 6)
dual → tri	divide by 3/2 — n × 2/3	6 → 4
tri → quad	divide by 4/3 — n × 3/4	4 → 3
quad → dual	n × 2	3 → 6

The result is floored at 1 so the ladder can never reach zero.

Why quad → dual multiplies

This is the one place the implementation departs from the literal brief. The brief says the quad peak is "divided by 2". Taken literally the ladder collapses: dual 4 → tri 2 → quad 1 → dual 0 inside a single lap. The other two transitions both scale by the ratio of stream counts, and quad → dual under that same rule is 4/2, which is *multiplying* by 2. That closes the loop exactly: dual 6 → tri 4 → quad 3 → dual 6. A player who does not improve returns to where they started; a player who does improve returns higher. The literal halving remains available as `literalQuadToDualHalving: true`.

7. Excluding and restricting permutations

Exclusion works at two levels:

- **Option level.** Turn individual entries off — a particular audio set, image set, or 2D/3D itself. Turning most options off and leaving a few on is how an allow-list is expressed: only the surviving combinations enter rotation.
- **Permutation level.** Block one exact combination by id while leaving its components available to other combinations.

Because a stranded rotation would deadlock the mode, the selection is validated before it can be saved. All of the following must hold:

- at least one position mode is enabled;
- at least one audio set is enabled;
- at least one colour, shape or image set is enabled — tri needs one;
- two *different* extra families have at least one set each — quad needs a pair;
- each of the three tiers still contains at least one permutation.

8. What the simulation shows

Replaying the scheduler with quad-box's real auto-progression thresholds (advance above 80%, drop below 50%) over 120 simulated days gives roughly **4.3 days per permutation** and **29 distinct stimulus configurations played**. Isolating the plateau rule from auto-progression, across 40 seeded runs of 365 days:

Setting	Mean days per block	Blocks ending at the minimum
relative 15% (default)	3.2	88%
relative 10%	3.2	82%
relative 5%	3.4	70%
absolute +15 points	3.1	90%
relative 15%, 4-day window	4.2	0%

Block length is governed by the window, not the threshold. Dropping the bar from 15% to 5% moves the mean from 3.2 to 3.4 days. The two conditions overlap heavily: a player whose n-level has been flat for three days is, almost by definition, not improving by 15% either. If blocks feel too short, the lever is the number of flat-n days required — raising it to 4 lifts the mean to 4.2 days. The threshold slider is close to inert.

Whether that is a problem depends on intent. A roughly three-day rotation does deliver "never let them stay comfortable" quite literally. It is only worth changing if you wanted permutations to persist long enough for a player to consolidate one before moving on.

9. State and persistence

Everything the scheduler needs is one serialisable object:

Field	Holds
cycleIndex	Position in the dual / tri / quad loop
pools	Permutation ids already used, per tier
lastDrawn	Previous permutation per tier, for the refill guard
current	Active block: permutation, n-level, day log, variant pool
history	Completed blocks with the reason each one ended
enabled / blocked	Exclusion settings
config	Thresholds, window, seed n-level, handoff mode

The scheduler is pure: no DOM, no storage, no timers. Persisting it is a single serialise-on-change, and the day log is what makes the plateau rule replayable and testable.

10. How it plugs into the app

Two new files carry the mode; the rest are small edits to existing ones.

File	Change
lib/constantChange.js	New — the whole scheduler. Pure, no DOM or storage.
stores/constantChangeStore.js	New — persistence, game-history reads, settings writes.
lib/ConstantChange.svelte	New — the settings panel.
stores/settingsStore.js	dual and quad presets replaced by one constant preset; default mode and enabled modes updated.
migrations/v4.js	New — moves existing players off dual/quad, inheriting their trial time and match chance.
lib/ModeSwapper.svelte	dual and quad removed, Constant Change added; unknown saved modes filtered out.
lib/GameSettings.svelte	Trials, interference, grid and the stimulus selects hidden — the scheduler owns them.
lib/Drawer.svelte	Feedback and rotation speed hidden for the same reason.
lib/DefaultGame.svelte	Syncs the schedule on mode select, on day rollover, and after each completed game.
lib/nback.js	Stamps the mode onto each game record.
lib/autoProgression.js	Matches on mode as well as title, so one mode cannot progress another.
lib/gamedb.js	addGame now waits for the transaction to commit.

Two fixes came out of the integration. Game records carried no mode, so a Custom A game with audio and colour and a Constant Change tri block were both titled "tri" and auto-progression could not tell them apart — one mode's results would move the other's n-level. And `addGame` resolved before its write committed: `await store.add(...)` only yields a microtask because an IDBRequest is not a promise, and `tx.complete` is not a real IndexedDB property. Anything reading straight afterwards could miss the game just played.

What the scheduler does not touch

N-back is seeded once when a block starts and then belongs to the app's existing auto-progression. Trial time and match chance are left alone. Feedback and rotation speed are app-wide settings in quad-box rather than per-mode, so Constant Change writes them at the start of each training day; switching to Custom A afterwards inherits whatever was last drawn.

Verification

`tests/constantChange.test.js` covers the permutation space, the pools, the plateau rule, the n-level handoff, the day fold and the settings projection — 52 tests, all passing. A separate replay drives the full loop against an in-memory game database and confirms the tier cycle never breaks, no block ends early, and the n-level stays inside the app's 1–12 range.